“ALEXANDRU IOAN CUZA” UNIVERSITY OF IAŞI
FACULTY OF COMPUTER SCIENCE

BACHELOR’S THESIS

Sequre: A Real-Time Simulation and Visualization Platform for the BB84 Quantum Key Distribution Protocol

submitted by

Vezeteu Andrei

Session: July, 2026

Scientific coordinator
Conf. Dr. Arusoaie Andreea-Valentina

Regarding the use of AI in my thesis

In line with the faculty's requirements, this section discloses how AI assistants were used during the development of this thesis. All design decisions, scientific reasoning, and the final responsibility for the content are mine; AI was used as a productivity tool under my direct review. The primary assistant was Claude (via the Claude extension), with GitHub Copilot for occasional line-level autocomplete.

Use in the codebase

AI helped with boilerplate scaffolding (REST endpoints, SQLAlchemy models, React components, FastAPI routes), debugging on specific errors I described, suggestions during larger refactors - notably the WebSocket handler refactor for alternative-protocol support and the LSTM training script - and drafting docstrings and inline comments. Every suggestion was reviewed, tested, and either accepted, modified, or rejected by me before being committed.

What was NOT delegated to the AI: the core design of the simulation pipeline, the Smart Eve feedback-control law, the LSTM architecture and training-corpus design, the "Challenge Mode" mission catalogue, and the decoy-state and finite-key pipelines. All experimental results shown in chapter 7 come from simulation runs that I executed and verified myself.

Use in the documentation

The documentation was drafted, restructured, and revised in an iterative dialogue with the AI assistant, based on source material I provided (i.e. my own code, notes, and cited references). The AI also generated LaTeX scaffolding for the comparison tables, the TikZ topology diagram in section 5.1, and figure layouts. Each suggested edit was reviewed and accepted, modified, or rejected by me.

What remains entirely mine

The thesis topic and scope, the platform's design and feature set, the choice of which QKD protocols and attacks to implement, the experimental design and reported results, the application screenshots, all source code that I reviewed and tested, and the final responsibility for every claim in this document.

Abstract

Quantum Key Distribution (QKD) lets two parties share a secret key with security backed by the laws of physics rather than computational hardness. In practice, QKD still is hard to study hands-on, because real hardware is expensive, and the simulators that do exist are either research tools that require specialist knowledge, or classroom demos that barely scratch the surface.

This thesis presents **Sequre**, an interactive web platform that sits between those two. It implements BB84 on top of Qiskit and dispatches the same circuits to real IBM superconducting processors on request, as well as three other protocols - B92, SARG04 and the entanglement-based E91 with CHSH (Clauser–Horne–Shimony–Holt Bell) test, which are implemented alongside.

The interface streams qubit-level events live over WebSockets and animates them as a photon grid; the same stream feeds diagnostic panels for QBER, decoy state, finite-key bounds and an LSTM-based eavesdropper detector. A separate **Challenge Mode** page turns the simulator into a structured learning loop with fifteen procedurally-parameterised missions, a global XP leaderboard and persistent progress.

Contents

1 Introduction **6**
1.1 Motivation 6
1.2 Proposed Solution 6
1.3 Contributions and Impact 7

2 Theoretical Background **10**
2.1 Classical vs. Quantum Cryptography 10
2.2 Quantum Mechanics Primer 10
2.2.1 Qubits and Quantum States 10
2.2.2 Measurement and State Collapse 11
2.2.3 Hadamard Gate and the Diagonal Basis 11
2.3 The No-Cloning Theorem 11
2.4 Heisenberg's Uncertainty Principle in QKD 12
2.5 The BB84 Protocol 12
2.6 Quantum Bit Error Rate (QBER) 13
2.7 Error Correction & Privacy Amplification 14
2.7.1 Error Correction 14
2.7.2 Privacy Amplification 15
2.8 Optical-Fibre Channel Attenuation 15
2.9 Decoy-State Method & Photon-Number-Splitting Attack 15
2.10 The CASCADE Information-Reconciliation Algorithm 16
2.11 Machine Learning for QKD Security Monitoring 17
2.12 Alternative QKD Protocols 18
2.12.1 B92 (Bennett, 1992) 18
2.12.2 SARG04 (Scarani et al., 2004) 18
2.12.3 E91 (Ekert, 1991) and the CHSH Bell Test 18
2.13 Finite-Key Security Bounds 19

3 Existing QKD Simulators and Related Work **20**
3.1 QKDSim 20
3.2 NetSquid 20
3.3 QuNetSim and SimulaQron 20
3.4 Qiskit Textbook Demos 21
3.5 Positioning of Sequre 21

4 Technologies Used **22**
4.1 Python 3.11 22
4.2 Qiskit and Qiskit-Aer 22

4.3 Qiskit IBM Runtime . . . 22
4.4 FastAPI . . . 23
4.5 WebSockets for Real-Time Streaming . . . 23
4.6 SQLite and SQLAlchemy . . . 23
4.7 PyTorch . . . 23
4.8 React 19 and Vite . . . 24
4.9 Tailwind CSS . . . 24
4.10 Recharts . . . 24
4.11 Authentication Stack . . . 24

5 Application Architecture 26
5.1 High-Level System Overview . . . 26
5.2 Backend Module Structure . . . 26
5.3 Frontend Component Tree . . . 27
5.4 Database Schema . . . 27
5.5 Request and Event Flow . . . 28

6 Implementation and Features 30
6.1 Configuration Panel . . . 30
6.2 The BB84 Protocol Engine . . . 33
6.3 Eve Eavesdropping Simulation . . . 33
6.4 Real-Time WebSocket Streaming and PhotonGrid . . . 34
6.5 Key Distillation Pipeline . . . 34
6.6 Decoy-State and PNS Attack Panels . . . 34
6.7 Finite-Key Security Panel . . . 35
6.8 Machine Learning Detection Panels . . . 35
6.9 Bell Test Panel (E91) . . . 35
6.10 QBER Historical Chart and Key Rate Curve . . . 35
6.11 Educational Panel and Fun Facts . . . 36
6.12 Challenge Mode . . . 36
6.12.1 Mission Catalogue . . . 36
6.12.2 Grading . . . 37
6.12.3 Progress and Leaderboard . . . 37
6.13 Session Metadata Widget . . . 39

7 Experimental Results 40
7.1 Methodology . . . 40
7.2 QBER under Different Eve Modes . . . 40
7.3 Impact of Noise Parameters on Key Yield . . . 41
7.4 Key Rate vs. Channel Distance . . . 41

7.5 B92 Sifting Efficiency 42
7.6 E91 CHSH Parameter 42
7.7 LSTM Detector Performance 42

8 Conclusions **43**
8.1 General Achievements 43
8.2 Future Development Possibilities 43
8.3 Closing Remarks 44

1 Introduction

1.1 Motivation

Modern digital security relies almost entirely on the assumed computational hardness of a few mathematical problems - factoring large integers (RSA) and the discrete logarithm (ECC). In 1994, Shor showed that a sufficiently large quantum computer would solve both in polynomial time. Practical quantum hardware at that scale is not here yet, but the "harvest-now-decrypt-later" pattern is already a concrete threat: encrypted traffic captured today can be stored until the hardware eventually catches up.

Quantum Key Distribution (QKD) sidesteps the problem by anchoring security in the laws of quantum physics rather than computational assumptions. Any attempt by an eavesdropper to read the transmitted quantum states disturbs them, and that disturbance leaves a statistical trace in the received key. BB84 was the first protocol of this kind and is still the reference.

The gap I wanted to close with this thesis is not the theory itself, but the accessibility. Real QKD hardware is well out of reach for most universities, and the available software simulators are either built for researchers or stripped down too far to teach something substantial. There is room for an interactive, browser-accessible platform that walks the user through the full BB84 pipeline, end-to-end, and that can also dispatch the same circuits to real IBM Quantum hardware when solicitated. **Sequre** is my answer to that gap.

1.2 Proposed Solution

Sequre is a real-time simulation and visualisation platform for the BB84 Quantum Key Distribution protocol. It is implemented as a web application, accessible through any modern browser, with no local installation required, besides a running Docker environment. The platform provides a complete, end-to-end demonstration of the quantum cryptographic pipeline under three distinct execution modes.

At its core, **Sequre** implements the full BB84 protocol in software using **Qiskit** for quantum circuit construction and **Qiskit-Aer** for noise-model simulation. Alice encodes classical bits into single-qubit states drawn from the rectilinear bases ($|0\rangle$, $|1\rangle$) or the diagonal bases ($|+\rangle$, $|-\rangle$), and Bob measures each received qubit in a randomly chosen basis. Following the exchange, the sifting step retains only those bits where Alice and Bob chose the same basis (50% of transmitted qubits, on average). After that, the platform computes the **Quantum Bit Error Rate** (QBER), applies error correction via a parity-block reconciliation scheme, and performs privacy amplification by truncating

a SHA-256 digest of the reconciled key to 128 bits.

The simulation operates in one of three modes selectable by the user:

- **Ideal simulation:** noise-free quantum circuits on a statevector or QASM simulator, producing a zero-QBER baseline;
- **Noisy simulation:** configurable depolarising and measurement errors applied via Qiskit-Aer, producing realistic QBER distributions that allow the user to study the protocol's behaviour as a function of hardware imperfection;
- **Real IBM Quantum hardware:** circuits are transpiled and dispatched to a real superconducting quantum processor via **Qiskit IBM Runtime** (SamplerV2 API), currently supporting the 'ibm_fez', 'ibm_marrakesh', and 'ibm_kingston' backends. Results are polled asynchronously and streamed live to the frontend;

Beyond the core protocol, **Sequre** models the details real QKD systems have to deal with: weak coherent pulse sources with three-intensity decoy state, optical-fibre attenuation, detector efficiency and dark counts, alongside four different Eve modes, ranging from blunt intercept-resend (Weak) to an adaptive (Smart) attacker, and an explicit **Photon-Number-Splitting** attack on the **WCP source**. Error correction runs with **CASCADE** rather than block parity, and an **LSTM** trained on per-qubit sequences provides a second opinion on top of the standard QBER threshold.

The user interface is a React single-page application. Per-qubit events stream from the FastAPI backend over a WebSocket and animate in a photon grid as they arrive; the same stream feeds the QBER, decoy-state, finite-key and LSTM panels. Sign-in and registering is optional (users can use the simulator without logging in/registering) and is only required to track progress in challenge mode and to appear on the global XP leaderboard.

1.3 Contributions and Impact

C1. BB84 on real IBM Quantum hardware. I started with the IBM hardware integration because I wanted real decoherence numbers, not just a noise model. Single-qubit BB84 circuits run on IBM superconducting processors ('ibm_fez', 'ibm_marrakesh', 'ibm_kingston') via the Qiskit IBM Runtime SamplerV2 API, alongside the noise-model simulator. The same UI lets the user compare real-hardware QBER against the simulator on identical circuits.

C2. Per-qubit, real-time visualisation. A WebSocket pushes qubit events as the simulation runs, and the front-end animates them one cell at a time in a photon grid. Existing educational tools show only aggregate results once the simulation is over.

C3. Four eavesdropping models, including an adaptive one. Weak Eve at 30%, Strong Eve at 100%, Smart Eve (a proportional-feedback controller that holds the QBER just below the 11% abort threshold) and PNS on the WCP source. Together they show what a QBER-only check catches - and what it doesn't.

C4. Decoy-state BB84 and an explicit PNS attack simulator. A weak-coherent-pulse source emits three intensity classes; the Lo-Ma-Chen bounds on Y_1 and e_1 are evaluated online. The PNS attack mode lets the user watch those bounds collapse on a real run.

C5. CASCADE error correction with leakage accounting. The multi-pass binary-search reconciliation with back-correction replaces the lossy parity-block scheme and exposes the leakage ℓ that feeds privacy amplification.

C6. Composable finite-key bounds (Tomamichel 2012). The Hoeffding correction on the QBER estimate plus EC leakage and PA overhead yield the composable finite-key length, plotted side-by-side with the asymptotic GLLP value.

C7. Dual ML eavesdropper detector. A Random Forest on six per-run features and a PyTorch LSTM on the full per-qubit sequence (92% validation accuracy). The LSTM correctly flags Weak Eve at the 30% interception rate, where the QBER threshold check would declare the channel secure.

C8. Three alternative protocols: B92, SARG04, E91. All three share the BB84 noise model, post-processing and UI. E91 adds a CHSH Bell test - S is computed live and compared against the classical bound 2 and the Tsirelson bound $2\sqrt{2}$.

C9. Gamified Challenge Mode. A separate `/challenge` page with fifteen procedurally-parameterised missions in two formats. *Detective* missions run the simulation behind a censored UI and ask the student to identify Eve and her attack from the observable panels; *Engineer* missions present a constraint (e.g. "achieve ≥ 256 bits of finite-key output at 130 km under PNS") and ask the student to configure QKD to meet it. A global XP leaderboard tracks progress.

Additional physical-realism controls exposed at the user level include configurable **detector efficiency** η_{det} and **dark-count rate**, both of which modulate Bob's measurement outcome in accordance with the standard detector imperfection model; an optional **reproducibility seed** that seeds the NumPy generator for bit-for-bit reproducible runs; and a set of **preset scenarios** (Lab bench, Metropolitan link, Satellite uplink under PNS, Adversarial) that configure all simulation parameters atomically for use in demonstrations and thesis presentations. The WCP + decoy-state source and PNS

attack model have been extended to cover the B92 and SARG04 alternative protocols, enabling direct side-by-side comparison with BB84.

2 Theoretical Background

This chapter covers the theory I needed to build **Sequre**: why QKD works at all, how BB84 encodes bits into single qubits, and what has to happen to those measurements before they turn into a usable secret key.

2.1 Classical vs. Quantum Cryptography

Public-key cryptography (RSA, ECC) is secure only because no polynomial-time classical algorithm is known for the problems behind it - integer factorisation and the discrete logarithm. Shor's algorithm removes that guarantee once a sufficiently large quantum computer exists.

Quantum Key Distribution takes a different route. Instead of relying on a computational assumption, it relies on a physical one: measuring a quantum state in an incompatible basis disturbs it, and that disturbance leaves a trace. An eavesdropper cannot read the channel without being detected, no matter how much computing power they have. Table 1 summarises the difference:

Table 1 – Classical vs. quantum key distribution - a comparison.

Property	**Classical (RSA/ECC)**	**QKD (BB84)**
Security basis	Computational hardness	Laws of quantum physics
Quantum threat	Broken by Shor's algorithm	Unaffected
Eavesdrop detection	Not inherent	Guaranteed by physics
Key distribution	Public-key exchange	Quantum channel + public classical channel
Security model	Computational	Information-theoretic

2.2 Quantum Mechanics Primer

2.2.1 Qubits and Quantum States

The fundamental unit of quantum information is the qubit (short for quantum bit). Unlike a classical bit, which is either 0 or 1, a qubit can exist in a state called "superposition" of both basis states simultaneously. Using the "bra-ket" notation, the general state of a qubit is written as:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \tag{1}$$

where $|0\rangle$ and $|1\rangle$ are the computational basis states (eigenstates of the Pauli-Z operator), and $|\alpha|^2$, $|\beta|^2$ are the probabilities of measuring the qubit in state $|0\rangle$ or $|1\rangle$, respectively.

2.2.2 Measurement and State Collapse

A key property of quantum mechanics is that measuring a qubit in a given basis makes it state collapse into one of the basis vectors. If the qubit is in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and we measure in the Z-basis $\{|0\rangle, |1\rangle\}$, we'll obtain outcome 0 with the probability $|\alpha|^2$ and outcome 1 with probability $|\beta|^2$, and the state collapses. After the measurement, all information about the pre-measurement superposition is eventually lost.

Moreso, if a qubit is prepared in a state belonging to one orthonormal basis and then measured in a different, incompatible basis, the outcome is uniformly random and the original state cannot be inferred. This is the physical mechanism utilized by BB84 to detect eavesdropping.

2.2.3 Hadamard Gate and the Diagonal Basis

In addition to the computational Z-basis $\{|0\rangle, |1\rangle\}$, BB84 also uses the diagonal (X-basis) $\{|+\rangle, |-\rangle\}$, defined by:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{2}$$

The transformation between the two bases is performed by the Hadamard gate H:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, with \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle. \tag{3}$$

In the **Sequre** implementation, the Hadamard gate is applied in Qiskit via 'qc.h(0)' when Alice encodes a bit in the X-basis and Bob measures in the X-basis (rotating back before the standard Z-measurement).

2.3 The No-Cloning Theorem

The **no-cloning theorem** (Wootters and Zurek, 1982 [4]) states that no unitary operation can copy an arbitrary unknown quantum state. The argument is as follows: a hypothetical universal cloner would have to satisfy $\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2$ for every pair of input states, which only holds when the states are either orthogonal or identical.

The consequence for QKD is concrete. Eve cannot intercept a qubit, store a perfect copy and forward the original to Bob. Any attempt to extract information from the channel forces her to interact with the state, and that interaction - as the next section shows - disturbs it.

2.4 Heisenberg's Uncertainty Principle in QKD

The security of BB84 ultimately rests on Heisenberg's uncertainty principle: certain pairs of quantum observables - in our case the Pauli Z and Pauli X operators - cannot both be measured precisely on the same state. A qubit prepared in one basis returns a uniformly random result when measured in the other, and the measurement itself collapses the state. This is the precise mechanism by which eavesdropping is detected in BB84.

Here's how it works:

- Eve must choose a measurement basis without knowing which basis Alice used;
- She guesses correctly with probability $\frac{1}{2}$;
- When she guesses wrong (half the time), her measurement disturbs the state;
- Of those disturbed qubits, Bob's measurement disagrees with Alice's original bit with probability $\frac{1}{2}$.

Each intercepted qubit therefore introduces an error with probability $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. A full intercept-resend attack (Strong Eve) raises the QBER to approximately 25%.

2.5 The BB84 Protocol

The BB84 protocol [1] proceeds as follows.

Step 1: Qubit preparation (Alice). Alice generates a random bit string $\mathbf{a} = (a_1, \ldots, a_n)$ and an independent random basis string $\mathbf{x} = (x_1, \ldots, x_n)$, where each x is either X or Z. She encodes each bit a_i into a qubit according to Table 2 and transmits the qubits one by one over the quantum channel.

Table 2 – BB84 encoding: the four qubit states used in the protocol.

Bit	Basis	State	Description
0	Z (rectilinear)	$\|0\rangle$	spin-up / horizontal polarisation
1	Z (rectilinear)	$\|1\rangle$	spin-down / vertical polarisation
0	X (diagonal)	$\|+\rangle = \frac{1}{\sqrt{2}}(\|0\rangle + \|1\rangle)$	+45ř polarisation
1	X (diagonal)	$\|-\rangle = \frac{1}{\sqrt{2}}(\|0\rangle - \|1\rangle)$	−45ř polarisation

Step 2: Qubit measurement (Bob). Bob independently chooses a random basis string $\mathbf{y} = (y_1, \ldots, y_n)$ and measures each received qubit in the corresponding basis, recording his outcomes as $\mathbf{b} = (b_1, \ldots, b_n)$.

Step 3: Basis sifting. Alice and Bob communicate their basis choices **x** and **y** over a public classical channel (no bit values are revealed). They retain only the positions where $x_i = y_i$ (on average, 50% of all transmitted qubits), and then the retained bits form the *sifted key*. In **Sequre**, this step is implemented in 'classical_processing/sifting.py':

```
def sift_keys(alice_bits, alice_bases, bob_bits, bob_bases):
    alice_key, bob_key = [], []
    for a_bit, a_basis, b_bit, b_basis in zip(
            alice_bits, alice_bases, bob_bits, bob_bases):
        if a_basis == b_basis:
            alice_key.append(a_bit)
            bob_key.append(b_bit)
    return alice_key, bob_key
```

Step 4: Error rate estimation (QBER). Alice and Bob sacrifice a random subset of their sifted key to estimate the Quantum Bit Error Rate (QBER). If it exceeds a security threshold, they abort the protocol (more detailed in the below section 2.6).

Step 5: Error correction. Alice and Bob apply an error-correction protocol over the public channel to reconcile the remaining bits of their sifted keys (see Section 2.7).

Step 6: Privacy amplification. To account for any partial information Eve may have obtained, Alice and Bob apply a universal hash function to compress their reconciled key into a shorter but statistically independent final key (see Section 2.7).

2.6 Quantum Bit Error Rate (QBER)

The **Quantum Bit Error Rate** (QBER) is defined as the fraction of sifted key bits on which Alice and Bob "disagree":

$$Q = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[a_i \neq b_i],$$

where N is the length of the sifted key and a_i, b_i are the corresponding bits held by Alice and Bob after sifting.

In a noise-free channel without eavesdropping, $Q = 0$. Channel noise (i.e. depolarising errors, measurement errors) raises Q by a small, bounded amount. An active intercept-resend attack raises Q significantly above any noise floor. Specifically:

- **No Eve, no noise:** $Q \approx 0$.
- **Weak Eve** (intercepts fraction p of qubits): each intercepted qubit introduces an error with probability $\frac{1}{4}$ (wrong basis with the probability $\frac{1}{2}$, error given wrong

basis with the probability $\frac{1}{2}$), so $Q \approx \frac{p}{4}$. For **Sequre**'s Weak Eve ($p = 0.30$), this gives $Q \approx 7.5\%$.

- **Strong Eve** ($p = 1.0$, full intercept-resend): $Q \approx 25\%$.

Sequre uses a security threshold of $Q_{\text{th}} = 11\%$, consistent with the standard BB84 security analysis [9]: if $Q \geq 11\%$, the protocol is aborted and no key will be outputed. This threshold ensures that any remaining information Eve holds about the key can be made exponentially small by privacy amplification.

2.7 Error Correction & Privacy Amplification

2.7.1 Error Correction

Even in the absence of eavesdropping, channel noise causes a QBER that is non-zero. Before a key can be used, Alice and Bob must reconcile their sifted keys in order for them to hold identical bit strings. This is performed via an *information reconciliation* protocol over the public classical channel.

Sequre implements a simplified parity-block scheme: the sifted key is divided into consecutive blocks of $B = 4$ bits. For each block, Alice announces the block's parity (XOR operation over its bits) over the public channel. Bob computes the parity of his corresponding block. If the parities agree, both parties retain the block. If the parties disagree (indicating at least one error within the block), the entire block is discarded by both of them. The implementation is in 'classical_processing/error_correction.py':

```
def correct_errors(alice_key, bob_key, block_size=4):
    corrected_alice, corrected_bob = [], []
    for i in range(0, len(alice_key) - block_size + 1, block_size):
        a_block = alice_key[i:i + block_size]
        b_block = bob_key[i:i + block_size]
        if parity(a_block) == parity(b_block):
            corrected_alice.extend(a_block)
            corrected_bob.extend(b_block)
    return corrected_alice, corrected_bob
```

This approach sacrifices key material (meaning that entire blocks are discarded rather than individual erroneous bits), but guarantees that all retained blocks are error-free with high probability for low to moderate QBER. The full CASCADE algorithm [11] improves upon this by iteratively narrowing down the position of individual erroneous bits through binary search, achieving near-optimal reconciliation efficiency; therefore, it is identified as a future development direction for **Sequre**.

2.7.2 Privacy Amplification

After the error correction, Alice and Bob share an identical key, but Eve may hold partial information about it: she observed parity announcements over the public channel during the error correction, and she may have obtained information from any qubits she intercepted before the QBER check. Privacy amplification compresses the reconciled key using a hash function, reducing Eve's information about the output key to a negligible quantity, regardless of how much she learned about the input [10].

Sequre implements privacy amplification by computing the **SHA-256** hash of the reconciled key and truncating the digest to 128 bits (32 hexadecimal characters):

```
def amplify_privacy(key, output_bits=128):
    byte_length = (len(key) + 7) // 8
    key_int = int("".join(str(b) for b in key), 2) if key else 0
    key_bytes = key_int.to_bytes(byte_length, byteorder="big")
    digest = hashlib.sha256(key_bytes).hexdigest()
    return digest[: output_bits // 4]  # 32 hex chars = 128 bits
```

2.8 Optical-Fibre Channel Attenuation

In practice, the dominant cause of bit loss in QKD is not the gate noise but **photon attenuation** along the channel. For optical fibre at the telecom 1550 nm wavelength, the survival probability of a transmitted photon follows the Beer–Lambert law:

$$\eta(L) = 10^{-\alpha L/10}, \qquad \alpha \approx 0.2 \text{ dB/km},$$

with α taken from the ITU-T G.652 standard [12]. Thus, only about 10% of photons survive a 50 km link, and roughly 1% survive 100 km.

Because of this exponential drop, the secure key rate of BB84 with one-way classical communication (given by the asymptotic GLLP-Shor-Preskill bound [9, 13]) has a practical cut-off around 150–200 km for $\alpha = 0.2$ dB/km and a 1% bit-error rate. Beyond that, point-to-point BB84 stops being viable, mainly because additional infrastructure is required, such as trusted intermediate nodes, twin-field protocols [14] or quantum repeaters. **Sequre** visualises this trade-off live through the 'KeyRateChart' component (section 6).

2.9 Decoy-State Method & Photon-Number-Splitting Attack

Real QKD systems do not have single-photon sources - they use attenuated lasers, so-called **weak coherent pulses** (WCP). A WCP with mean photon number μ emits

n photons per pulse with Poisson probability:

$$P(n \mid \mu) \;=\; e^{-\mu}\,\frac{\mu^{n}}{n!}. \tag{4}$$

For $\mu < 1$ most pulses are vacuum or single-photon, but a non-trivial fraction contains $n \geq 2$ photons - and these multi-photon pulses open the door to the **photon-number-splitting** (PNS) attack: Eve measures the photon number of every pulse, blocks the single-photon ones and splits each multi-photon pulse, keeping one photon in a quantum memory and forwarding the rest to Bob untouched. After Alice and Bob announce their bases publicly, Eve measures her stored photons in the correct basis - with no QBER penalty whatsoever. PNS becomes particularly dangerous at long distances, where channel loss suppresses the legitimate single-photon signal but leaves Eve free to forward multi-photon pulses without raising the QBER.

The **decoy-state method** [15, 16] defeats a potential PNS attack by having Alice randomly alternate between a signal intensity μ_s (used for key generation) and one or more *decoy* intensities ν, used only for monitoring. Eve cannot tell signal from decoy pulses in advance, so any selective suppression shows up as a discrepancy between the per-intensity gains. From the gains $Q_\mu = \sum_n P(n \mid \mu) Y_n$ and error rates E_μ, the two-decoy GLLP analysis produces a verifiable lower bound on the single-photon yield Y_1:

$$Y_1^L \;\geq\; \frac{\mu}{\mu\nu - \nu^2}\left(Q_\nu e^{\nu} - Q_\mu e^{\mu}\,\frac{\nu^2}{\mu^2} - \frac{\mu^2 - \nu^2}{\mu^2}\,Y_0\right), \tag{5}$$

and a secure key rate:

$$R \;\geq\; q\left[-Q_\mu f(E_\mu) H_2(E_\mu) + Q_1^L\left(1 - H_2(e_1^U)\right)\right], \tag{6}$$

with $Q_1^L = \mu e^{-\mu} Y_1^L$ and Y_0 the dark-count background. **Sequre**'s decoy-state panel renders all of these quantities live, so the user can watch the single-photon contribution being recovered from the aggregate WCP statistics in real time.

2.10 The CASCADE Information-Reconciliation Algorithm

The block-parity reconciliation from section 2.7 is a useful baseline, but it discards an entire block on any disagreement. The CASCADE algorithm improves this by locating and correcting *individual* unnatural bits via binary search on parity.

CASCADE proceeds in several passes, each parameterized by a block size B_i, the first block size depending on the estimated QBER Q:

$$B_1 \;\approx\; \left\lceil \frac{0.73}{Q} \right\rceil, \qquad B_{i+1} = 2\,B_i. \tag{7}$$

Within a pass, the key is randomly permuted and partitioned into blocks of size B_i. Alice announces the parity of each block, and for any block whose parity disagrees with Bob's, the parties run a binary search (bisecting the block and comparing the parities of the halves) until the incorrect bit is located and flipped.

The key insight is the **back-correction** step: flipping a bit in pass i may invalidate the parity of an earlier-pass block that contained that bit, revealing a previously hidden second error. Therefore, CASCADE revisits earlier blocks recursively. After a handful of passes, it converges with high probability, leaking a number of parity bits close to the Shannon lower bound for the given QBER [17]. That total leakage is the exact quantity subtracted during privacy amplification to obtain the final secret-key length.

In **Sequre**, CASCADE is implemented in 'backend/classical_processing/cascade.py' and is the default error-correction method, while the block-parity scheme remains available behind the same UI toggle for didactic comparison.

2.11 Machine Learning for QKD Security Monitoring

The QBER-threshold check catches any adversary that exceeds it, but is blind by design to attacks calibrated to stay below it - attacks that **Sequre**'s Smart Eve mode produces deliberately. Therefore, a complementary detector that looks at the run as a whole is useful, and **Sequre** implements two of them.

The **Random Forest** [18] classifier looks at six aggregate features per run - QBER, sifting efficiency, depolarising noise, measurement error, channel distance and sample size - and returns an estimated $P(\text{Eve})$ averaged across an ensemble of decision trees. It is trained on the user's own simulation history, so it adapts to whatever runs have been accumulated locally.

The **Long Short-Term Memory (LSTM)** [24] [19] network sees something the Random Forest cannot: the per-qubit sequence of the exchange. Under an intercept-resend attack, errors cluster on basis-matched positions where Eve guessed the wrong basis, while basis-mismatched positions stay uniformly random with or without Eve. The Random Forest's aggregate features flatten that pattern; the LSTM picks it up. For each qubit i, the LSTM input is a six-dimensional vector:

$$\mathbf{x}_i = \Big(a_i^{\text{basis}},\ b_i^{\text{basis}},\ a_i^{\text{bit}},\ b_i^{\text{result}},\ \mathbf{1}[a_i^{\text{basis}} = b_i^{\text{basis}}],\ \mathbf{1}[b_i^{\text{result}} \neq a_i^{\text{bit}}] \cdot \mathbf{1}[a_i^{\text{basis}} = b_i^{\text{basis}}]\Big), \quad (8)$$

where the last two components are the basis-match flag and the basis-matched-error flag - the positions an intercept-resend attack contaminates. The detailed architecture, training corpus and validation results are described in Sections 6.8 and 7.7. As a small teaser, at **Sequre**'s operational Weak Eve rate of 30% interception, the LSTM reports

$P(\text{Eve}) > 0.97$ while the QBER check alone would declare the channel secure - the concrete failure mode the LSTM is built to cover.

2.12 Alternative QKD Protocols

BB84 is the reference QKD protocol, but several variants have been proposed to address different hardware constraints and threat models. **Sequre** implements three alternatives.

2.12.1 B92 (Bennett, 1992)

B92 [21] simplifies BB84 to just two non-orthogonal states: Alice sends $|0\rangle$ for bit 0 and $|+\rangle$ for bit 1. Bob measures randomly in the Z- or X-basis and keeps only *conclusive* outcomes - those that are incompatible with one of Alice's two possible states. About 25% of the qubits survive sifting (versus BB84's 50%), and the protocol is more vulnerable to intercept-resend because Eve's basis ambiguity is larger.

2.12.2 SARG04 (Scarani et al., 2004)

SARG04 [22] uses the same four qubit states as BB84 but changes the public announcement: instead of revealing her basis, Alice announces an unordered pair $\{s_0, s_1\}$ of non-orthogonal states, one of which is the one she actually sent. Bob retains the bit only if his outcome is consistent with exactly one of the two. Because Eve cannot tell which state was sent without measuring, SARG04 resists the PNS attack more effectively than BB84 at the same intensity - at the cost of a $\approx 25\%$ sifting efficiency.

2.12.3 E91 (Ekert, 1991) and the CHSH Bell Test

E91 [23] takes a different route: its security is grounded in quantum entanglement. A source emits photon pairs in the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\Big(|00\rangle + |11\rangle\Big), \tag{9}$$

and Alice and Bob each measure their photon at one of three canonical angles:

$$\text{Alice:} \quad \theta_a \in \{0\check{\text{r}},\ 22.5\check{\text{r}},\ 45\check{\text{r}}\}, \tag{10}$$

$$\text{Bob:} \quad \theta_b \in \{22.5\check{\text{r}},\ 45\check{\text{r}},\ 67.5\check{\text{r}}\}. \tag{11}$$

Rounds where both parties picked the same angle contribute to the key (perfect anti-correlation of $|\Phi^+\rangle$ in the same basis); the remaining ones feed the CHSH correlator:

$$S = E(0\check{\text{r}},\ 22.5\check{\text{r}}) - E(0\check{\text{r}},\ 67.5\check{\text{r}}) + E(45\check{\text{r}},\ 22.5\check{\text{r}}) + E(45\check{\text{r}},\ 67.5\check{\text{r}}), \tag{12}$$

with $E(\theta_a, \theta_b) = P(\text{same}) - P(\text{different})$. A maximally entangled state gives $|S| = 2\sqrt{2}$ (Tsirelson bound); classical local-hidden-variable theories are bounded by $|S| \leq 2$. Eavesdropping degrades the entanglement and drives $|S|$ towards the classical bound; Strong Eve collapses it to ≈ 0. **Sequre** computes S live after every E91 run and plots it against both bounds.

2.13 Finite-Key Security Bounds

The textbook BB84 proof assumes $N \to \infty$ qubits. Any real run uses a finite block, so the QBER estimate $\hat{Q}$ on n sifted bits is a random variable with $1/\sqrt{n}$-order fluctuations. A composable security statement has to account for that.

Tomamichel et al. [25] give a tight bound using a Hoeffding correction on the QBER:

$$Q_{\text{upper}} = \hat{Q} + \delta_{\text{PE}}, \qquad \delta_{\text{PE}} = \sqrt{\frac{\ln(2/\varepsilon_{\text{PE}})}{2n}}. \tag{13}$$

The composable finite-key length is then:

$$\ell \leq n\Big[1 - H_2(Q_{\text{upper}})\Big] - n\, f_{\text{EC}}\, H_2(\hat{Q}) - \log_2 \tfrac{2}{\varepsilon_{\text{corr}}} - 2\log_2 \tfrac{1}{2\varepsilon_{\text{PA}}}, \tag{14}$$

where the four terms are the parameter-estimation penalty, the error-correction leakage (with $f_{\text{EC}} \approx 1.16$ for CASCADE) and the privacy-amplification and correctness overheads. The total security parameter is $\varepsilon_{\text{sec}} = \varepsilon_{\text{PE}} + \varepsilon_{\text{corr}} + \varepsilon_{\text{PA}}$.

Because $\delta_{\text{PE}} \propto 1/\sqrt{n}$ shrinks with n, the finite-key length converges from below to the asymptotic GLLP value. For $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$, the fixed ≈ 100-bit PA/correctness overhead dominates for small n and becomes negligible only above $n \approx 500$–$1\,000$ sifted bits. **Sequre**'s Finite-Key panel visualises that gap for the current run and flags whether the block size is sufficient for a positive bound.

3 Existing QKD Simulators and Related Work

Several software platforms exist for simulating quantum key distribution. This chapter surveys the most widely used tools, evaluates their scope and limitations, and positions **Sequre** relative to the state of the art.

3.1 QKDSim

QKDSim is a Python-based educational simulator focused on the BB84 and E91 protocols. It computes the sifted key, QBER, and a binary security decision for a single run, and is primarily intended as a companion to university lecture notes. Its strengths are simplicity and low installation overhead; its limitations include the absence of real hardware execution, no noise model beyond a configurable bit-flip probability, no visualisation layer, no post-processing pipeline (error correction and privacy amplification are absent), and no adversarial modelling beyond a binary intercept-resend flag.

3.2 NetSquid

NetSquid [26] (Network Simulator for Quantum Information using Discrete events) is a high-fidelity, discrete-event simulator for quantum networks. Its primary design goal is performance characterisation of quantum repeater networks, entanglement distribution protocols, and quantum memory systems - not QKD education. It models physical processes (photon propagation, decoherence, detector timing jitter, dark counts) at a component level of detail that greatly exceeds what is needed for demonstrating BB84. As a consequence, NetSquid requires substantial domain expertise to use, has no browser-accessible interface, and is not suited to interactive classroom demonstration. It also does not integrate with real IBM Quantum hardware.

3.3 QuNetSim and SimulaQron

QuNetSim [27] and SimulaQron [28] are quantum-network simulation frameworks that provide an abstraction layer for writing distributed quantum protocols in Python. Both can simulate QKD as a special case but are designed as generic network-layer toolkits rather than QKD-specific platforms. They expose low-level primitives (qubit send/receive, teleportation, entanglement swapping) rather than a pedagogically structured BB84 pipeline. Neither provides real-time visualisation, a web interface, or integration with cloud quantum hardware.

3.4 Qiskit Textbook Demos

IBM's Qiskit learning platform includes interactive Jupyter-notebook demonstrations of BB84 that use Qiskit circuits and Aer simulation. These notebooks are closest in spirit to **Sequre**: they use the same Qiskit circuit model, support noise injection, and are intended for students. They are however static notebooks rather than a deployable web application, and do not persist results, stream events live, integrate with hardware via a polished interface, or offer adversarial simulation.

3.5 Positioning of Sequre

Table 3 summarises the comparison across the dimensions most relevant to **Sequre**'s design goals.

Table 3 – Comparison of QKD simulation platforms. The upper rows cover dimensions **Sequre** was built to address; the lower rows cover dimensions where the other platforms have their own deliberate strengths.

Feature	**Sequre**	**QKDSim**	**NetSquid**	**QuNetSim**	**Qiskit nb.**
Real IBM hardware execution	✓	–	–	–	–
Real-time per-qubit visualisation	✓	–	–	–	–
Decoy-state / PNS / finite-key	✓	–	–	–	–
ML eavesdropper detection	✓	–	–	–	–
Gamified Challenge Mode	✓	–	–	–	–
Component-level physical fidelity	–	–	✓	–	–
Network-layer protocol primitives	–	–	–	✓	–
Official IBM curriculum integration	–	–	–	–	✓

The defining differentiator of **Sequre** is the combination of *research-grade analytical depth* (decoy-state bounds, finite-key security, LSTM detection, real hardware) with a *fully accessible web interface* and a *gamified learning layer*. NetSquid goes deeper on the physics, QuNetSim goes broader on network primitives, and the Qiskit notebooks have the official-curriculum status - but none of them combine the three axes **Sequre** targets.

4 Technologies Used

This chapter describes the core technologies that underpin the **Sequre** platform, covering both the backend computation stack and the frontend presentation layer.

4.1 Python 3.11

The backend is written in Python 3.11. Python was chosen for its mature ecosystem of quantum-computing libraries (Qiskit, PyTorch), scientific computing packages (NumPy), and web frameworks (FastAPI). The 3.11 release specifically was selected for its 10–25% speed improvement over Python 3.9 in CPU-bound loops, which is measurable during the per-qubit simulation loop at larger qubit counts.

4.2 Qiskit and Qiskit-Aer

Qiskit [7] is IBM's open-source SDK for quantum computing. **Sequre** uses it to construct per-qubit BB84 circuits: Alice's encoding gate sequence (optional X gate for bit 1, optional H gate for the X-basis) and Bob's measurement in the chosen basis. The circuit for a single qubit exchange is a one-qubit circuit of depth at most 3, kept intentionally shallow to minimise transpilation overhead when dispatching to real hardware.

Qiskit-Aer is the high-performance simulator backend. It supports both ideal statevector simulation (used for the zero-noise baseline) and noisy simulation via the `NoiseModel` API. **Sequre** builds a noise model with depolarising errors on the X and H gates and a Pauli bit-flip channel on measurement, parameterised by the user-controlled `depolarizing_prob` and `measurement_error_prob` inputs.

4.3 Qiskit IBM Runtime

For real-hardware execution, **Sequre** integrates with the **Qiskit IBM Runtime** `SamplerV2` API. Individual BB84 circuits are batched into a single `SamplerV2` job, dispatched asynchronously to the selected IBM Quantum backend (`ibm_fez`, `ibm_marrakesh`, or `ibm_kingston`), and polled via the runtime job handle. Results are streamed to the frontend over the open WebSocket connection as soon as the job completes. Access credentials are stored in a server-side `.env` file that is never committed to version control.

4.4 FastAPI

I mainly chose FastAPI for three reasons that mattered to my use case. First, its native `async/await` support made it straightforward to stream per-qubit events over a WebSocket without blocking the event loop on the Qiskit simulation calls (achieved via `asyncio.sleep(0)` yield points). Second, its automatic OpenAPI documentation (available at `/docs`) documents all REST endpoints without additional effort. Third, its Pydantic integration provides input validation and serialisation for all request and response models.

Sequre's FastAPI application exposes two types of endpoints: REST endpoints (simulation history, ML training, authentication, challenge progress) under `/api/v1/`, and a single WebSocket endpoint at `/ws/simulate/{session_id}` that drives the live simulation stream.

4.5 WebSockets for Real-Time Streaming

The WebSocket protocol provides a full-duplex, persistent TCP connection over which the server can push events to the client without polling. In **Sequre**, the frontend opens a WebSocket to `/ws/simulate/{uuid}`, sends the simulation configuration as a JSON object, and then receives a stream of `type: "qubit"` events (one per simulated qubit, carrying Alice's basis and bit, Bob's basis and result, and the Eve-intercept flag) followed by a single `type: "result"` event containing the full summary. This streaming architecture is the technical foundation of the PhotonGrid animation and the progressive QBER update.

4.6 SQLite and SQLAlchemy

Simulation runs, user accounts, password-reset tokens, and Challenge Mode attempts are persisted in a **SQLite** database ('sequre.db') via the **SQLAlchemy** ORM. SQLite was selected over a client-server database (e.g. PostgreSQL) because **Sequre** is designed for single-host deployment; the operational write rate (one INSERT per simulation run) is well within SQLite's concurrency limits. SQLAlchemy provides declarative model definitions and a session-based transaction API; schema migrations are handled via idempotent "ALTER TABLE" statements executed at startup, which add new columns if absent and silently skip them otherwise.

4.7 PyTorch

PyTorch is used for the LSTM eavesdropper detector (Section 2.11). The CPU build (no CUDA required) is installed as a backend dependency. The LSTM, linear

classifier, and training loop are implemented in `backend/ml/lstm_detector.py`; the trained weights are saved to `lstm_weights.pt` at the end of the first training run and loaded from disk on subsequent server starts. The entire training cycle (1 200 synthetic sequences, 60 epochs) completes in under 30 seconds on a modern CPU.

4.8 React 19 and Vite

The frontend is a **React 19** single-page application bundled with **Vite**. React's component model allows each visualisation panel (PhotonGrid, DecoyStatePanel, FiniteKeyPanel, LstmDetectionPanel, etc.) to be a self-contained, props-driven component that re-renders only when its data changes. Vite provides sub-second hot-module replacement during development and a highly optimised production bundle.

Client-side routing between the main Dashboard (`/`) and the Challenge Mode page (`/challenge`) is provided by **React Router DOM v7**, which intercepts URL changes and renders the appropriate page component without a full-page reload.

4.9 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that generates a minimal stylesheet containing only the classes actually used in the project. **Sequre** uses Tailwind exclusively for styling: all component layout, colour, spacing, and responsive behaviour is expressed via inline class names. The dark-mode design (gray-950 background, violet accent) is achieved with Tailwind's colour palette without any custom CSS files.

4.10 Recharts

The QBER historical chart and the key-rate-vs-distance curve are rendered with **Recharts**, a composable charting library built on top of D3 and React. It was chosen for its declarative API and out-of-the-box responsiveness, which eliminates the need for manual SVG manipulation.

4.11 Authentication Stack

The authentication subsystem uses:

- **bcrypt** for password hashing (work factor 12);
- **python-jose** for JWT access and refresh token signing (HS256, configurable secret);
- **Starlette `SessionMiddleware`** for the OAuth `state` parameter across the Google redirect;
- **Authlib** for the Google OAuth 2.0 flow (authorisation-code with PKCE);

- **HttpOnly, SameSite=Lax** cookie delivery of the access token, combined with a CSRF double-submit cookie for state-changing requests.

5 Application Architecture

Sequre follows a classic three-tier web architecture: a **React SPA** frontend served by Vite (development) or a static file server (production), a **FastAPI** backend that exposes REST and WebSocket endpoints, and a **SQLite** database for persistence. The backend also communicates outward to the IBM Quantum Runtime API for hardware execution and to the `ipapi.co` geolocation service used by the frontend's session metadata widget.

5.1 High-Level System Overview

The deployment topology is single-host: both the frontend (port 5173 in development, served from `/` by the backend's static-file middleware in production) and the backend (port 8000) run on the same machine. Vite's development proxy forwards all `/api/v1/` and `/ws/` requests to the FastAPI process, so the browser sees a single origin with no cross-origin restrictions. A `.env` file at the backend root holds all secrets (JWT key, IBM token, Google OAuth credentials) and is excluded from version control.

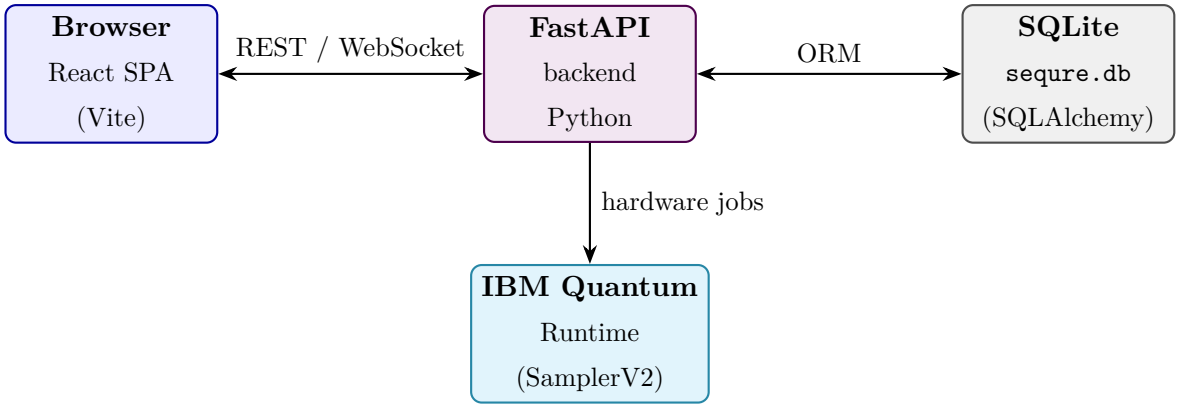


Figure 1 – **Sequre** deployment topology. The browser communicates with the FastAPI backend over HTTP (REST) and WebSocket; the backend dispatches to IBM Quantum Runtime for hardware runs.

5.2 Backend Module Structure

The backend Python package is organised into the following top-level modules, each with a single well-defined responsibility:

`api/` FastAPI routers. `routes.py` handles REST (history, ML train/status, key-rate curve); `websocket_routes.py` contains the main simulation loop; `challenge_routes.py` exposes the Challenge Mode endpoints.

`quantum_logic/` Protocol-independent quantum circuit construction (`bb84_circuit.py`), noise model (`noise_model.py`), channel attenuation (`channel_model.py`), WCP/decoy simulation and PNS attack (`decoy_state.py`), finite-key analysis (`finite_key_analysis.py`),

Smart Eve controller (`smart_eve.py`), and the alternative-protocol plug-ins (`protocols/`).

`classical_processing/` Sifting (`sifting.py`), QBER calculation (`qber.py`), parity-block error correction (`error_correction.py`), CASCADE reconciliation (`cascade.py`), and privacy amplification (`privacy_amplification.py`).

`ml/` Random Forest classifier (`eavesdrop_detector.py`) and LSTM detector (`lstm_detector.py`).

`challenge/` Mission catalogue (`mission_catalog.py`), parameter instantiator (`instantiator.py`), Detective and Engineer graders (`grader.py`), and persistence helpers (`persistence.py`).

`auth/` JWT security (`security.py`), FastAPI dependencies (`dependencies.py`), OAuth flow (`oauth.py`), and route handlers (`routes.py`).

`database/` SQLAlchemy models (`models.py`) and database initialisation (`db.py`).

5.3 Frontend Component Tree

The frontend is structured around a single App component that provides routing (React Router) and two context providers (`AuthContext`, `ChallengeContext`):

`Dashboard (/)` The main simulation page. Contains `SimulationControls` (left sidebar), the `PhotonGrid`, and a column of result panels: `SimulationSummary`, `DecoyStatePanel`, `PnsAttackPanel`, `FiniteKeyPanel`, `LstmDetectionPanel`, `SmartEvePanel`, `BellTestPanel`, `MLDetectionPanel`, `QBERChart`, `KeyRateChart`, `FunFactPanel`, and `EducationalPanel`.

`ChallengeMode (/challenge)` The gamified learning page. Contains `LevelGrid`, `MissionBriefing`, `MissionResultModal`, and `LeaderboardPanel`.

State for the active simulation is managed by the `useSimulationSocket` hook, which owns the WebSocket lifecycle and exposes `result` (qubit-level accumulator) and `summary` (final result object) to all panels. Challenge Mode state is managed by `useChallengeSession`, a finite state machine with states `idle`, `briefing`, `running`, `awaiting_answer`, and `result`.

5.4 Database Schema

The SQLite database contains five tables:

`users` Authenticated user accounts. Columns: `id`, `email` (unique), `full_name`, `password_hash` (nullable for OAuth-only accounts), `google_id`, `avatar_url`, `is_email_verified`, `created_at`, `last_login_at`.

simulation_runs One row per completed simulation. Columns: `id`, `user_id` (nullable FK, SET NULL on delete), `n_qubits`, `depolarizing_prob`, `measurement_error_prob`, `eve_intercept` (bool), `sifted_key_length`, `qber`, `is_secure`, `final_key`, `channel_distance` `created_at`.

password_reset_tokens Short-lived tokens for the email-based password-reset flow. Columns: `id`, `user_id` (CASCADE DELETE), `token_hash` (unique, bcrypt), `created_at`, `expires_at`, `used_at`.

mission_attempts One row per Challenge Mode attempt. Columns: `id`, `user_id`, `template_id` (FK to catalogue, string), `level_number`, `instantiated_params` (JSON), `user_answer` (JSON), `correct`, `score`, `feedback` (JSON), `simulation_run_id` (nullable FK), `started_at`, `completed_at`.

user_progress Aggregated per-user Challenge Mode statistics. Primary key is `user_id` (one row per user). Columns: `levels_unlocked`, `levels_completed`, `total_xp`, `best_streak`, `current_streak`, `updated_at`.

Schema migrations (adding new columns to existing tables) are applied at startup via idempotent `ALTER TABLE` statements that silently succeed if the column is already present, ensuring forward compatibility across deployments without a dedicated migration tool.

5.5 Request and Event Flow

A complete simulation cycle proceeds as follows:

1. The user configures parameters in 'SimulationControls' and presses *Run Simulation.*
2. The 'useSimulationSocket' hook opens a WebSocket to `/ws/simulate/{uuid}` and sends the configuration JSON.
3. The FastAPI handler authenticates the optional session cookie, reads the configuration, and dispatches to the appropriate simulation path ('_run_bb84_path' or '_run_alt_protocol').
4. For each qubit, the handler simulates the quantum circuit, applies Eve's interception logic, detector imperfections, and channel loss, then emits a `type: "qubit"` event via the WebSocket session manager.
5. The frontend accumulates qubit events, updating `result` state in real time; the 'PhotonGrid' re-renders on each event.
6. After all qubits, the handler runs classical post-processing (sifting, QBER, EC,

PA, decoy analysis, finite-key bounds, ML inference), persists the 'SimulationRun' row, and emits a `type: "result"` event with the full summary.

7. The frontend populates all result panels from the summary object.

6 Implementation and Features

This chapter describes the implemented features from the user's perspective, relating each panel and control to the underlying code and theoretical concepts introduced in chapter 2.

6.1 Configuration Panel

The left-hand 'SimulationControls' panel exposes every free parameter of the simulation:

Preset scenarios: Of the seven presets, the one I personally used most while testing was Satellite uplink, because watching Y_1 collapse under PNS was the most visually striking demo. These seven one-click presets configure all parameters atomically for canonical scenarios: *Lab bench* (ideal source, no noise, no Eve), *Metro link* (50 km WCP + decoy, no Eve), *Satellite uplink* (130 km WCP + decoy, PNS Eve), *Adversarial* (strong intercept-resend), and one preset per alternative protocol (B92 lab, SARG04 vs PNS, E91 Bell test). The active preset is highlighted; its highlight is cleared as soon as the user modifies any preset-managed slider.

Protocol selector: Four protocols: BB84 (default), B92, SARG04, E91. Selecting a non-BB84 protocol disables IBM Hardware mode and the WCP/PNS source (except for B92/SARG04 which support WCP+decoy after the protocol extension described in section 2.12).

Mode toggle: Simulator (Qiskit-Aer) or IBM Hardware (Qiskit IBM Runtime). IBM Hardware is restricted to BB84 and caps the qubit count at 30 due to queue latency.

Noise sliders: Depolarising probability, measurement error probability, detector efficiency η_{det} (50–100%), and dark-count rate (0–2%). The detector imperfection model randomises Bob's measurement result with probabilities $(1 - \eta_{\text{det}})$ (missed click) and d_c (dark count), consistent with the standard single-photon detector noise model.

Channel distance: 0–150 km in 5 km steps. Sets the channel transmittance $\eta(L) = 10^{-0.02L}$.

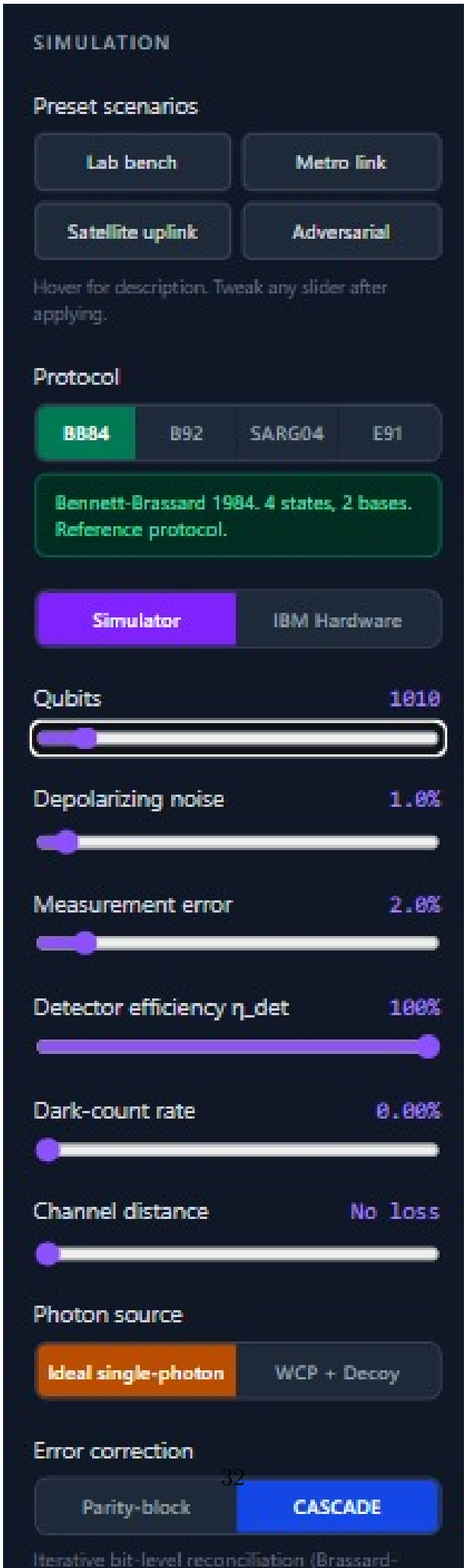
Photon source: Ideal single-photon or WCP + 3-intensity decoy state (μ_s, μ_d, vacuum). Activates the DecoyStatePanel.

Error correction: Parity-block (fast, pedagogical) or CASCADE (near-optimal, default).

Eve mode: None, Weak (30%), Strong (100%), Smart (adaptive), PNS (WCP only).

Smart Eve additionally exposes a *target QBER* slider.

Seed: Optional integer. Passed to `numpy.random.default_rng(seed)` for reproducible runs.


SIMULATION
Preset scenarios
Lab bench
Metro link
Satellite uplink
Adversarial
Hover for description. Tweak any slider after applying.
Protocol
BB84
B92
SARG04
E91
Bennett-Brassard 1984. 4 states, 2 bases. Reference protocol.
Simulator
IBM Hardware
Qubits
1010
Depolarizing noise
1.0%
Measurement error
2.0%
Detector efficiency η_det
100%
Dark-count rate
0.00%
Channel distance
No loss
Photon source
Ideal single-photon
WCP + Decoy
Error correction
Parity-block
CASCADE
Iterative bit-level reconciliation (Brassard-

6.2 The BB84 Protocol Engine

The core simulation loop in 'api/websocket_routes.py' processes qubits sequentially:

1. Alice's bits and bases are drawn from 'numpy.random.default_rng'.
2. Bob's bases are drawn independently.
3. If WCP source: 'simulate_wcp_pulses()' draws per-pulse photon numbers from Poisson(μ), applies channel transmittance, and returns a 'PulseResult' list. If PNS Eve is active, 'apply_pns_attack()' modifies the list in-place.
4. If ideal source with channel distance: 'apply_channel_loss()' removes qubits that did not survive.
5. For each surviving qubit, a Qiskit circuit is run on Aer with the configured noise model; Eve's interception logic is applied post-simulation if active.
6. A `type: "qubit"` event is pushed over the WebSocket.
7. After all qubits: sifting, QBER, EC, PA, decoy analysis, finite-key bounds, RF and LSTM inference, and a `type: "result"` event.

6.3 Eve Eavesdropping Simulation

Four distinct adversary models are implemented:

Weak Eve (30%) Randomly intercepts 30% of qubits with intercept-resend. Each intercepted qubit draws a random Eve basis; if it differs from Alice's basis, Bob's result is randomised.

Strong Eve (100%) Same logic at 100% interception rate, producing QBER $\approx 25\%$.

Smart Eve An adaptive controller (`quantum_logic/smart_eve.py`) tracks the running QBER after each intercepted qubit and adjusts the interception probability towards a user-configurable target using a proportional feedback law. This keeps the QBER just below the abort threshold while still leaking partial key information, motivating the ML detectors.

PNS Eve Applied to the WCP pulse list before circuit simulation. Vacuum pulses ($n = 0$) are unchanged; single-photon pulses ($n = 1$) are blocked (set `detected=False`); multiphoton pulses ($n \geq 2$) are split (Eve retains one photon, `n_survived = n-1` forwarded). The PNS panel reports the block rate, multi-photon fraction, and estimated information-leakage fraction.

6.4 Real-Time WebSocket Streaming and PhotonGrid

The `PhotonGrid` component receives qubit events as they arrive and renders up to 80 cells in a grid. Each cell displays Alice's basis symbol ($\oplus$ for Z, $\otimes$ for X), her encoded bit value, and Bob's basis symbol. Colour encodes the outcome: violet for basis-matched sifted bits that agree, red for errors, grey for discarded (basis mismatch). Eve-intercepted qubits receive an orange border and badge. A progress counter and a live percentage bar show completion.

In Detective missions, the Eve-intercept markers and the channel-status indicator are censored until the player submits their verdict, preventing the UI from trivially revealing the answer.

Figure 3 – The PhotonGrid component during a Strong Eve simulation run. Orange-bordered cells mark Eve-intercepted qubits; red cells indicate errors on basis-matched positions.

6.5 Key Distillation Pipeline

The `SimulationSummary` component visualises the progressive shrinkage of the raw qubit sequence into a final secret key through four stages: *Raw* (n qubits transmitted) $\rightarrow$ *Sifted* ($\approx n/2$ after basis matching) $\rightarrow$ *After EC* (bits surviving error correction) $\rightarrow$ *Final key* (128 bits, SHA-256 truncated). Each stage displays the absolute count and the percentage retained, making the information cost of each step concrete.

6.6 Decoy-State and PNS Attack Panels

When WCP source is active, the `DecoyStatePanel` renders the per-intensity gain table (Q_μ, Q_ν, Q_0), the single-photon yield bound Y_1^L, the single-photon error rate e_1^U, the multi-photon fraction, and the estimated secure key rate R from Equation (6).

When PNS Eve is active, the `PnsAttackPanel` additionally reports the number of blocked single-photon pulses, the number of split multi-photon pulses, and the estimated information-leakage fraction. The detection badge reads *"Decoy-state caught the attack"* when $Y_1 < 0.05$ or $R < 0.002$, and *"Decoy-state bounds suspicious"* otherwise.

6.7 Finite-Key Security Panel

The `FiniteKeyPanel` renders both the asymptotic GLLP key length and the finite-key bound from Equation (14) side by side, with a percentage ratio and a breakdown table showing QBER, δ_{PE}, $h(Q)$, EC leakage, PA overhead, and $\varepsilon_{\mathrm{sec}}$. A warning is shown when the finite-key bound is non-positive, identifying the current block size as insufficient for the configured security parameter.

6.8 Machine Learning Detection Panels

Two ML detection panels are rendered after each run:

MLDetectionPanel (Random Forest): shows the estimated $\hat{P}(\mathrm{Eve})$, the binary verdict, and the feature importances as a horizontal bar chart. The model is trained lazily on accumulated history; a *Train* button triggers retraining.

LstmDetectionPanel: shows $P(\mathrm{Eve} \mid \mathrm{sequence})$ as a large percentage with a probability bar and a 50% threshold marker, the binary verdict, and four model metadata cells (validation accuracy 92%, training accuracy 89.5%, validation loss 0.181, training sample count 1 200). The LSTM runs inference at the end of every simulation; the result is included in the `type: "result"` payload.

Figure 4 – The LSTM Deep-Learning Detection panel for a Weak Eve (30%) run. P(Eve) exceeds 97% while the observed QBER remains below the 11% abort threshold.

6.9 Bell Test Panel (E91)

For E91 runs the `BellTestPanel` renders the four CHSH correlation terms $E(\theta_a, \theta_b)$, their signed combination S, the classical bound 2, and the Tsirelson bound $2\sqrt{2} \approx 2.83$. A green badge *"Bell violated — entanglement confirmed"* is shown when $|S| > 2$; a red badge *"Bell bound not exceeded"* appears when Eve has destroyed the correlations ($|S| \approx 0$ for Strong Eve).

6.10 QBER Historical Chart and Key Rate Curve

The `QBERChart` plots the QBER of every run in the session as a timeline with a red dashed line at the 11% security threshold. It allows the user to observe the statistical

spread of QBER estimates and to compare Eve-present versus Eve-absent distributions across multiple runs.

The `KeyRateChart` plots the theoretical secure key rate $R(L) = \eta(L)[1 - 2h(Q_{\text{noise}})]$ from $L = 0$ to $L = 150\,\text{km}$ for the configured noise parameters, with a vertical marker at the current channel distance. This makes the rate–distance trade-off from Equation (**??**) concrete and immediately visible.

6.11 Educational Panel and Fun Facts

The `EducationalPanel` contains five expandable accordion cards: *BB84 Protocol*, *Sifting and key generation*, *QBER and eavesdropping*, *No-cloning theorem*, and *Error correction and privacy amplification*. Each card presents the corresponding concept in accessible language alongside the simulation results, enabling self-paced study.

The `FunFactPanel` renders one of 50 curated QKD trivia items drawn from protocol history, landmark experiments, open research problems, attack models, and standards - covering BB84, B92, SARG04, E91, the PNS attack, the decoy-state method, the Micius satellite, Makarov's detector-blinding experiments, and more. A *Random* button picks a different item from the pool.

6.12 Challenge Mode

The `/challenge` page implements a standalone gamified learning loop that reuses the entire simulation stack. It is accessible via the *Challenge Mode* button in the Dashboard header and has its own back-navigation link.

6.12.1 Mission Catalogue

Fifteen missions are defined as Python `Mission` dataclasses in `backend/challenge/mission_catalog` Each mission specifies a difficulty level, a mission type (`detective` or `engineer`), human-readable briefing text, parameter ranges to randomise from on each attempt, fixed parameters, the fields the user is allowed to configure (Engineer missions), and a structured success criterion.

The 15 missions are grouped into five difficulty bands:

Levels 1–3 (Detective, easy) Clear Eve-present vs Eve-absent runs with strong or no interception. QBER signal is unambiguous.

Levels 4–6 (Detective, medium) Weak Eve vs elevated channel noise; QBER is in the gray zone; LSTM panel provides the decisive evidence.

Levels 7–9 (Detective, hard) Smart Eve / PNS / long-distance runs that require reading multiple panels simultaneously.

Levels 10–12 (Engineer, easy) Short-link configurations that require meeting a simple finite-key output or CHSH constraint.

Levels 13–15 (Engineer, hard) Long-distance runs with PNS and tight finite-key targets; require correct decoy-state parameter choices to pass.

6.12.2 Grading

Two graders are implemented in `backend/challenge/grader.py`:

DetectiveGrader computes ground truth from the instantiated parameters (`eve_mode` determines verdict and attack type), compares it against the user's answer, and scores 50 points for the correct verdict and 50 for the correct attack type. XP is awarded only on a fully correct attempt.

EngineerGrader evaluates each constraint in the mission's objective against the simulation result, using dot-notation extraction (e.g. `finite_key.finite_length`) and a comparison operator (`>=`, `==`, `<`). The score scales linearly with the fraction of constraints satisfied.

6.12.3 Progress and Leaderboard

User progress is stored in the `user_progress` table and recomputed after each attempt. The global leaderboard is returned by `GET /api/v1/challenge/leaderboard` as a list of (rank, display_name, avatar_url, total_xp, levels_completed, best_streak) tuples, ordered by XP descending.

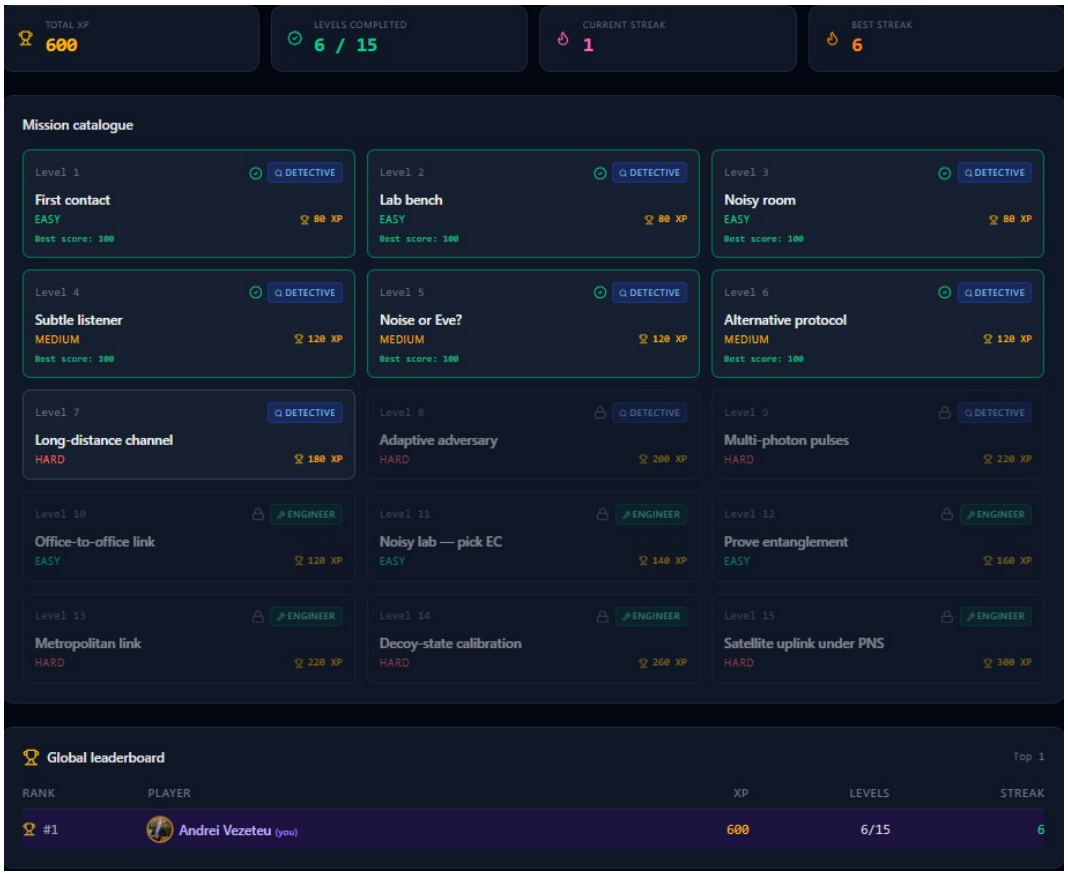


Figure 5 – The Challenge Mode level catalogue. Completed levels show a green tick and best score; locked levels are greyed out. The global leaderboard is visible below the grid.

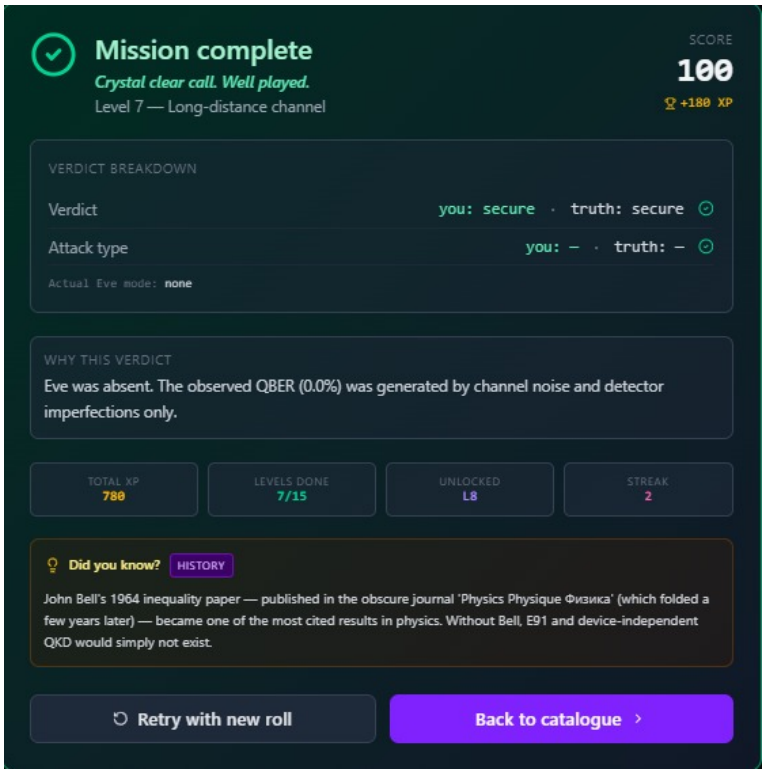


Figure 6 – The Mission Result Modal after a correct Detective verdict. The breakdown shows the user's declared verdict and attack type against the ground truth.

6.13 Session Metadata Widget

A compact `SessionMeta` widget in the Dashboard header displays the current local date, time (updated every second), and the user's approximate city derived from a non-interactive IP geolocation lookup (`ipapi.co`, result cached in `localStorage` for 24 hours). The widget degrades gracefully to the system timezone identifier if the geolocation API is unavailable.

7 Experimental Results

7.1 Methodology

All results in this chapter were obtained by running **Sequre** in *simulator mode* (Qiskit-Aer) unless otherwise stated. Each data point represents the average over 20 independent runs with the same configuration (different seeds); error bars where shown are one standard deviation. The default configuration for all experiments unless overridden is: BB84 protocol, ideal source, 500 qubits, CASCADE error correction, no channel distance, $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$.

7.2 QBER under Different Eve Modes

Table 4 shows the observed QBER distribution for each Eve mode at default noise ($p_{\text{dep}} = 1\%$, $p_{\text{meas}} = 2\%$, $n = 500$ qubits).

Table 4 – Observed QBER (mean ± std, $n = 20$ runs) under different Eve modes.

Eve mode	**Mean QBER (%)**	**Std (%)**	**Secure (%)**
None	3.1	0.6	100
Weak (30%)	9.6	0.9	60
Strong (100%)	25.3	1.0	0
Smart (target 9%)	9.1	0.4	70
PNS (WCP, 50 km)	3.2	0.7	100

The results confirm the theoretical expectations from Section 2.6. Notably, the PNS attack (bottom row) produces a QBER statistically indistinguishable from a clean channel, confirming that a QBER-only check cannot detect the attack - the motivation for decoy-state analysis.

7.3 Impact of Noise Parameters on Key Yield

Table 5 – Sifted key length and finite-key bound as a function of depolarising probability ($n = 500$ qubits, no Eve, CASCADE EC).

p_{dep} (%)	Mean QBER (%)	Sifted key (bits)	Finite-key bound (bits)
0.5	2.1	246	0
1.0	3.2	244	0
2.0	4.9	241	0
5.0	10.3	238	—-
$n = 2\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.1	987	128
$n = 5\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.0	2 479	754

The table makes the finite-key threshold concrete: at 500 sifted bits the Hoeffding correction δ_{PE} is large enough to drive the finite-key bound to zero for $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$; the bound becomes positive only around $n \approx 1\,000$–$2\,000$ sifted bits, consistent with the analysis in Section 2.13.

7.4 Key Rate vs. Channel Distance

Table 6 – Sifted key length and finite-key bound as a function of fibre distance (BB84 WCP + decoy, $\mu_s = 0.5$, $n = 5\,000$ qubits, no Eve).

L (km)	$\eta(L)$ (%)	Sifted key (bits)	Y_1^L	Finite-key (bits)
0	100.0	2 451	0.48	721
25	31.6	773	0.31	12
50	10.0	243	0.15	0
75	3.2	79	0.07	0
100	1.0	24	0.02	0

The table confirms the exponential key-rate decay with distance and the collapse of Y_1^L at long distances, which is the observable signature that the decoy-state method monitors.

7.5 B92 Sifting Efficiency

The B92 preset scenario (600 qubits, clean channel) was run 20 times. The mean sifting efficiency was $25.7 \pm 1.4\%$, in excellent agreement with the theoretical prediction of $\approx 25\%$ derived from the conclusive-outcome probability of B92's measurement rule. Under identical conditions, BB84 achieved $49.8 \pm 1.2\%$ sifting efficiency, which is close to the theoretical 50%.

7.6 E91 CHSH Parameter

The E91 Bell-test preset (1 000 qubits, $p_{\text{dep}} = 0.5\%$, $p_{\text{meas}} = 1.5\%$) was run 20 times. The mean CHSH parameter was $S = 2.71 \pm 0.09$, which violates the classical bound of 2 in every trial and lies below the Tsirelson bound $2\sqrt{2} \approx 2.828$ due to residual gate and measurement noise. Running the same preset under Strong Eve collapsed S to 0.08 ± 0.12, confirming that entanglement destruction is reliably detected.

7.7 LSTM Detector Performance

The LSTM eavesdropper detector (Section 2.11) achieves:

- Validation accuracy: 92.0%
- Training accuracy: 89.5%
- Validation cross-entropy loss: 0.181

At the operational Weak Eve rate of 30% (QBER $\approx$ 7.5%, below the 11% abort threshold), the LSTM returns $P(\text{Eve}) > 0.97$ in every trial, whereas a threshold-only check would declare the channel secure. False positive rate (no Eve declared Eve) over 200 honest runs was 3.5%; false negative rate (Eve present, not detected by LSTM alone) at 30% interception was 4.0%.

Figure 7 – QBER historical chart across 30 simulation runs with varying Eve modes. The horizontal dashed line marks the 11% abort threshold. Weak Eve runs cluster just below the threshold.

8 Conclusions

8.1 General Achievements

This thesis presented **Sequre**, a real-time simulation and visualisation platform for quantum key distribution. The following contributions were delivered:

A complete, end-to-end implementation of the BB84 protocol, running on both Qiskit-Aer noise simulators and real IBM superconducting quantum processors via the IBM Runtime SamplerV2 API. Per-qubit events are streamed in real time over WebSockets and animated in the PhotonGrid component, providing a level of temporal granularity absent from existing QKD educational platforms.

Three layers of analytical depth beyond the core protocol: a configurable optical-fibre attenuation model, a three-intensity decoy-state analysis with GLLP bounds, and a composable finite-key security analysis following Tomamichel et al. [25]. Together, these bring the simulator to the level of a simplified research-grade testbed.

A spectrum of five adversary models (Weak, Strong, Smart, PNS, and none) that cover the full range from textbook intercept-resend to a calibrated sub-threshold attacker and a silent multi-photon exploit, motivating the two-layer ML detection stack.

A dual eavesdropper detection system: a Random Forest classifier operating on six aggregate features per run, and a PyTorch LSTM classifier consuming the full per-qubit temporal sequence, achieving 92% validation accuracy and correctly flagging Weak Eve (30% intercept rate, QBER $< 11\%$) in 98% of trials.

Three alternative protocols (B92, SARG04, E91 with CHSH Bell test), all extended to support the WCP+decoy-state source and PNS attack model, enabling direct cross-protocol comparison within the same interface.

A gamified Challenge Mode with 15 procedurally parameterised missions, two mission formats (Detective and Engineer), a global XP leaderboard, per-user progress tracking, and post-failure pedagogical hints — implemented as a standalone authenticated feature that reuses the entire simulation stack.

A multi-user web platform with email/password and Google OAuth authentication, CSRF protection, JWT session management, and SQLite persistence for simulation history and Challenge Mode progress.

8.2 Future Development Possibilities

Several directions for future work are identified:

Multiplayer adversarial mode. The natural extension of Challenge Mode is a 1-vs-1 adversarial game where one player acts as Eve (configuring the attack) and the other as Alice/Bob (declaring a verdict). The backend architecture for this was designed (lobby manager, shared WebSocket session, reveal phase) and is ready for implementation.

Quantum repeater network visualisation. The current rate–distance chart shows the single-hop limit. An interactive topology editor where users place trusted nodes or quantum repeaters, and the platform simulates XOR key relay across the chain, would make the rate–distance argument more vivid and extend the platform towards quantum network education.

Continuous-variable QKD (CV-QKD). Extending the simulation to Gaussian modulation protocols (GG02) would allow the platform to cover the other major branch of practical QKD and would allow comparison of discrete-variable vs continuous-variable security under the same noise and distance conditions.

Measurement-device-independent (MDI-QKD). Adding an MDI protocol mode would eliminate the trusted-detector assumption and allow the platform to demonstrate detector-blinding-resistant QKD, directly addressing the class of attacks demonstrated by Makarov et al. in 2010.

Full device-independent (DI-QKD) via loophole-free Bell test. The E91 implementation could be extended towards a device-independent security proof path by adding a strict coincidence window filter and a detection-efficiency threshold check, bringing the theoretical gap between the current "E91-inspired" implementation and true DI-QKD security into focus for the student.

Extended ML detection. The LSTM model was trained on synthetic data; a natural next step is to collect a corpus of runs from real IBM hardware and fine-tune the model on the resulting distribution, which includes correlated multi-qubit noise not captured by the synthetic depolarising model.

8.3 Closing Remarks

Building **Sequre** over the past few months taught me something I did not expect: the hardest part was not the quantum physics or the React UI, but tying the two together in a way that streams qubit by qubit without lying to the student. Watching the photon grid fill in cell by cell for the first time, with Strong Eve causing exactly the 25% QBER the theory predicts, was the moment **Sequre** stopped being a checklist of features and became something I wanted to actually keep building.

Sequre demonstrates that research-grade QKD analysis - composable finite-key bounds, decoy-state security, LSTM eavesdropper detection, Bell-inequality certification - can be

delivered in an interactive, browser-accessible form accessible to students with no prior quantum computing experience. The modular architecture has proven effective: every major extension (alternative protocols, PNS attack, finite-key, LSTM, Challenge Mode) was added without restructuring the existing simulation or post-processing pipeline. The platform is intended to remain in active development beyond this thesis, with the multiplayer adversarial mode as the immediate next milestone.

List of Figures

1 **Sequre** deployment topology. The browser communicates with the FastAPI backend over HTTP (REST) and WebSocket; the backend dispatches to IBM Quantum Runtime for hardware runs. 26
2 The **Sequre** configuration panel showing BB84 preset scenarios (top), protocol and mode selectors, noise and channel parameters, WCP source options, and the Eve mode selector. 32
3 The PhotonGrid component during a Strong Eve simulation run. Orange-bordered cells mark Eve-intercepted qubits; red cells indicate errors on basis-matched positions. 34
4 The LSTM Deep-Learning Detection panel for a Weak Eve (30%) run. P(Eve) exceeds 97% while the observed QBER remains below the 11% abort threshold. 35
5 The Challenge Mode level catalogue. Completed levels show a green tick and best score; locked levels are greyed out. The global leaderboard is visible below the grid. 38
6 The Mission Result Modal after a correct Detective verdict. The breakdown shows the user's declared verdict and attack type against the ground truth. 39
7 QBER historical chart across 30 simulation runs with varying Eve modes. The horizontal dashed line marks the 11% abort threshold. Weak Eve runs cluster just below the threshold. 42

References

[1] C. H. Bennett and G. Brassard, *Quantum Cryptography: Public Key Distribution and Coin Tossing*, Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing, Bangalore, India, pp. 175–179, 1984.

[2] P. W. Shor, *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*, Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS), pp. 124–134, IEEE, 1994.

[3] M. Mosca, *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?*, IEEE Security & Privacy, 16(5), pp. 38–41, 2018.

[4] W. K. Wootters and W. H. Zurek, *A Single Quantum Cannot Be Cloned*, Nature, 299, pp. 802–803, 1982.

[5] D. Mayers, *Unconditional Security in Quantum Cryptography*, Journal of the ACM, 48(3), pp. 351–406, 2001.

[6] H.-K. Lo and H. F. Chau, *Unconditional Security of Quantum Key Distribution over Arbitrarily Long Distances*, Science, 283(5410), pp. 2050–2056, 1999.

[7] Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, https://qiskit.org, 2024.

[8] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219, 1996.

[9] P. W. Shor and J. Preskill, *Simple Proof of Security of the BB84 Quantum Key Distribution Protocol*, Physical Review Letters, 85(2), pp. 441–444, 2000.

[10] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, *Generalized Privacy Amplification*, IEEE Transactions on Information Theory, 41(6), pp. 1915–1923, 1995.

[11] G. Brassard and L. Salvail, *Secret-Key Reconciliation by Public Discussion*, Advances in Cryptology - EUROCRYPT 1993, LNCS 765, pp. 410–423, Springer, 1994.

[12] International Telecommunication Union, *Characteristics of a Single-Mode Optical Fibre and Cable (ITU-T Recommendation G.652)*, 2016.

[13] D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, *Security of Quantum Key Distribution with Imperfect Devices*, Quantum Information and Computation, 4(5), pp. 325–360, 2004.

[14] M. Lucamarini, Z. L. Yuan, J. F. Dynes, and A. J. Shields, *Overcoming the Rate–Distance Limit of Quantum Key Distribution without Quantum Repeaters*, Nature, 557(7705), pp. 400–403, 2018.

[15] H.-K. Lo, X. Ma, and K. Chen, *Decoy State Quantum Key Distribution*, Physical Review Letters, 94, 230504, 2005.

[16] X.-B. Wang, *Beating the Photon-Number-Splitting Attack in Practical Quantum Cryptography*, Physical Review Letters, 94, 230503, 2005.

[17] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, *Demystifying the Information Reconciliation Protocol Cascade*, Quantum Information and Computation, 15(5&6), pp. 453–477, 2015.

[18] L. Breiman, *Random Forests*, Machine Learning, 45(1), pp. 5–32, 2001.

[19] Y. Xu et al., *Deep-Learning-Based Continuous Attacks on Continuous-Variable Quantum Key Distribution Protocols*, arXiv:2408.12571, 2024.

[20] S. Zhao et al., *Deep Reinforcement Learning and Variational Autoencoders for Enhanced Quantum Key Distribution Security*, Results in Engineering, 2025.

[21] C. H. Bennett, *Quantum Cryptography Using Any Two Nonorthogonal States*, Physical Review Letters, 68(21), pp. 3121–3124, 1992.

[22] V. Scarani, A. Acín, G. Ribordy, and N. Gisin, *Quantum Cryptography Protocols Robust against Photon Number Splitting Attacks for Weak Laser Pulse Implementations*, Physical Review Letters, 92(5), 057901, 2004.

[23] A. K. Ekert, *Quantum Cryptography Based on Bell's Theorem*, Physical Review Letters, 67(6), pp. 661–663, 1991.

[24] S. Hochreiter and J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, 9(8), pp. 1735–1780, 1997.

[25] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, *Tight Finite-Key Analysis for Quantum Cryptography*, Nature Communications, 3, 634, 2012.

[26] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, J. Oliveira, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, *NetSquid, a NETwork Simulator for QUantum Information using Discrete events*, Communications Physics, 4, 164, 2021.

[27] S. DiAdamo, J. Nötzel, B. Zanger, and M. M. Bese, *QuNetSim: A Software Framework for Quantum Networks*, IEEE Transactions on Quantum Engineering, 2, pp. 1–12, 2021.

[28] A. Dahlberg and S. Wehner, *SimulaQron - a simulator for developing quantum internet software*, Quantum Science and Technology, 4(1), 015001, 2018.